Pattern Recognition 164 (2025) 111512



Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/pr





Iterative knowledge distillation and pruning for model compression in unsupervised domain adaptation☆

Zhiyuan Wang [a], Long Shi [a,*], Zhen Mei [a], Xiang Zhao [b], Zhe Wang [c], Jun Li [a,1]

[a] School of Electronic and Optical Engineering, Nanjing University of Science and Technology, Jiangsu, 210094, China
[b] College of Systems Engineering, National University of Defense Technology, Hunan, 410073, China
[c] School of Computer Science and Engineering, Nanjing University of Science and Technology, Jiangsu, 210094, China

ARTICLE INFO

Keywords:
Deep learning
Unsupervised domain adaptation
Adaptive channel pruning
Transfer knowledge distillation

ABSTRACT

In practical applications, deep learning models often face the challenges of inconsistent distribution between training data and test data and insufficient labeled data. To address these problems, unsupervised domain adaptation (UDA) based transfer learning has gained significant attention. However, the existing UDA models are difficult to meet the requirements of real-time and resource-constrained scenarios. Although model compression can accelerate UDA, it usually leads to performance degradation. In this paper, we propose an iterative transfer model compression (ITMC) method, which centers on two key modules, i.e., transfer knowledge distillation (TKD) and adaptive channel pruning (ACP), by executing them alternately. The tight coupling of the two modules realizes the effective compression of the model while ensuring the performance of the model on the target domain. In the TKD phase, the teacher model and the student model are gradually adapted to the target domain, and the real-time updated teacher model efficiently guides the student model learning, while the ACP phase employs a dynamic pruning strategy based on the training epoch, which removes unimportant channels based on the loss of the TKD student model. Experimental results demonstrate that ITMC approach achieves higher accuracy under the same compression ratio compared with the state-of-the-art methods.

1. Introduction

Deep learning (DL) has become a foundational technology in the field of artificial intelligence (AI). Among its tools, convolutional neural networks (CNNs) excel at feature extraction, enabling them to tackle complex tasks such as computer vision [1], natural language processing [2], and automatic speech recognition [3]. Ideally, CNNs are trained and evaluated on extensive labeled datasets, with the assumption that the training and testing data share a consistent distribution. However, in practical scenarios, acquiring sufficient labeled data for testing is often impractical, and the testing data frequently follows a distribution that differs from the training data. This distributional discrepancy leads to significant performance degradation when applied to datasets with unknown distributions (i.e., target domain).

One of the main solutions to this problem is transfer learning [4], which transfers the knowledge from the source domain to the target domain. Particularly, as a typical method of transfer learning, unsupervised domain adaptation (UDA) focuses on handling the cases where the target domain is unlabeled. Recently, a variety of innovative UDA methods have emerged to solve this problem, such as domain-adversarial training [5] and self-training [6]. These methods obviate the necessity for laboriously labeling a large amount of training data and thereby maximize the utilization of unlabeled data in the target domain.

Another problem is that with the exponential growth of data scale, neural network experiences a remarkable surge in both parameters and complexity. This consumes a large amount of computational resources and energy, which will limit the applicability of UDA in resource-constrained environments such as mobile devices, embedded systems, and edge computing. Driven by this issue, network compression has received much attention over recent years, since it can reduce the model size and ultimately decrease the computation and storage costs [7]. The classical compression methods include quantization [8,9], pruning [10–12], low-rank decomposition [13,14], and knowledge distillation (KD) [15–17]. In these methods, network quantization requires

☆ Part of this work has been presented in International Conference on Web Information Systems and Applications 2024 (WISA 2024) (Wang et al. 2024).

* Correspondence to: School of Electronic and Optical Engineering, Nanjing University of Science and Technology, Jiangsu Province, 210094, China
E-mail address: longshi@njust.edu.cn (L. Shi).
[1] School of Information Science and Engineering, Southeast University, Nanjing, China.

https://doi.org/10.1016/j.patcog.2025.111512
Received 25 May 2024; Received in revised form 15 December 2024; Accepted 24 February 2025
Available online 3 March 2025
0031-3203/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

specific hardware support, such as processors or accelerators that support low-bit numerical computation. Low-rank approximation relies on massive computational resources to decompose the parameters of the convolution kernel. Pruning removes redundant structures of the network that may cause severe performance degradation. KD compresses the network by distilling the knowledge from the teacher model to the student model. However, to preserve the performance, the selection of the student model is critical.

In light of the aforementioned constraints, researchers have also investigated the combination of multiple compression methods in order to harness complementary benefits of diverse methods. For example, some articles combined pruning and KD as a two-stage compression, e.g., pruning the student model first and then improving the performance of the pruned model through KD [18], or pruning the teacher model first and then distilling knowledge from pruned teacher model to the student model [19,20]. In [21], the unimportant connections of the model were first pruned, and then the remaining weight parameters were quantized. Furthermore, quantized distillation in [22] was proposed to add the quantization loss to the training of the student model. Additionally, the authors in [23] proposed a low-rank approximated channel pruning (LAP) framework to eliminate redundancy at the weight level and filter level.

In spite of these benefits, direct integration of these network compression approaches encounters some limitations. First, it is difficult to optimize these techniques individually to take full advantage of their potential. For example, the feature representation capability of the student model is limited due to the fact that pruning weakens the expressive power of the model. Second, it is rather challenging to apply network compression techniques to the problem of UDA. If network compression and UDA are regarded as two independent stages and combined straightforwardly, e.g., UDA after model compression, the compressed model may be limited in its representational ability and difficult in adapting to the complex features of the target domain. Conversely, if model compression is performed after UDA, it faces significant challenges due to the lack of labeled data in the target domain, which prevents the model from being fine-tuned to adapt to the target domain. How to effectively orchestrate the compression methods to balance the trade-off between the compression ratio and performance for UDA requires further exploration.

To address the aforementioned problems, we propose an iterative transfer model compression (ITMC) method by alternately performing transfer knowledge distillation (TKD) and adaptive channel pruning (ACP). The main objective is to compress deep neural network while simultaneously mitigating the impact of domain shifts. This work is an extended version of our conference paper [24] by providing in-depth analysis of TKD and ACP modules. Specifically, we delve into the design of the loss function and the importance of its components in the TKD module, as well as the channel evaluation criterion in the ACP module. To further validate our approach, we conduct experiments on multiple datasets and include additional ablation studies to demonstrate the impact of each component on model performance. In summary, the contributions of this paper are as follows:

- We propose an ITMC algorithm for UDA model compression. In each epoch, TKD and ACP are performed iteratively. In the TKD module, the teacher model and student model are co-trained to make the student model more adaptive to the target domain, while ACP is employed to further simplify the student model by pruning the redundant channels.
- The TKD module aims to improve the performance of the student model on the target domain through the guidance of the teacher model. A collaborative loss is designed to make the predictions of the student model consistent with those of the teacher model in both the source and target domains. Moreover, a trade-off factor is proposed to balance between fitting labeled data in the source domain and adapting unlabeled data in the target domain.
- To reduce the prediction difference before and after pruning, we develop a flexible pruning strategy for the student model in the ACP module, along with a pruning rate formula to determine whether the pruning should occur in each epoch. As the training progresses, the student model gradually adapts to the pruned model structure with the guidance of the teacher model.
- In our experiments, VGG16 and ResNet50 are employed to evaluate the proposed approach on three datasets, i.e., Office-31, ImageCLEF-DA and PACS. It is shown that ITMC achieves approximately 10% accuracy improvement compared with state-of-the-art model compression methods of UDA.

The rest of the paper is organized as follows. The UDA methods and model compression for UDA are introduced in Section 2 and the details of the proposed ITMC method are provided in Section 3. Our experimental results are described in Section 4. Finally, the paper is concluded in Section 5.

2. Related work

2.1. Unsupervised Domain Adaptation (UDA)

UDA is a transfer learning strategy that addresses the performance degradation problem caused by differences of data distribution between the source domain and unlabeled target domain. Generally, the UDA methods can be categorized into three types [25], i.e., sample-based UDA, mapping-based UDA, and adversarial-based UDA. First, sample-based UDA relies on samples with similar distributions across different domains [26]. The main issue is to identify samples with similar distributions in different domains and efficiently use them to enhance the performance in the target domain.

Second, mapping-based UDA aims to learn a mapping function that projects data from the source and target domains into a shared feature space [27]. In this way, the data from different domains have similar feature representations, thereby improving the generalization ability of the model on the target domain. Existing methods such as maximum mean discrepancy (MMD) [28] and correlation alignment (CORAL) [29] are commonly used to measure the feature differences between the source and target domain.

Third, adversarial-based UDA aims to reduce the domain difference between the source domain and target domain through adversarial training [30]. The method consists of three components, i.e., feature extractor, domain discriminator, and class predictor. First, a feature extractor generates useful features for both the source and target domains. Second, a domain discriminator makes it difficult to distinguish between source and target domain data features by maximizing the domain adversarial loss. Third, a class predictor maintains classification accuracy on the source domain. These three components are trained collaboratively to ultimately improve performance in the target domain.

2.2. Model compression for UDA

The combination of UDA and model compression is beneficial for creating lightweight and high-performance models that are more suitable for resource-constraint applications. In recent years, some works have explored the model compression techniques in UDA scenarios.

In [31], a soft-weighted MMD method was proposed to reduce the distribution difference between the source domain and the target domain, and the model pruning method based on the scaling factor in the batch normalization (BN) layer was used to reduce the model redundancy for partial domain adaptation (PDA). In [32], the authors introduced a method called transfer channel pruning (TCP) to compress the deep UDA model, wherein a transfer evaluation criterion based on the MMD loss was applied to identify and prune unimportant channels. However, these techniques require a pre-adapted model for the target

domain prior to pruning. Otherwise, the model accuracy may decrease significantly after pruning.

In [33], the authors utilized multiple teacher models for knowledge voting to generate target domain soft labels while distilling knowledge from multiple teacher models to student models. In [34], the authors proposed a progressive KD (PKD) that enables the student model to gradually learn domain-invariant features from the teacher model by dynamically adjusting the importance between UDA and PKD. In [35], the authors proposed an universal and joint knowledge distillation (UNI-KD) method to transfer source and target domain knowledge from the teacher model to the student model through adversarial learning. Since the structure of the student model is fixed, there is no guarantee that the student model can be free of redundancy after KD.

Through our literature review, we find that the existing methods on model compression for UDA still suffer from insufficient model compression and accuracy degradation. Furthermore, few attempts have been made to compress UDA models by jointly optimizing multiple model compression methods. In this paper, we propose ITMC to unleash the potential between complementary compression techniques.

3. Proposed ITMC method

3.1. Main idea

This section presents the proposed ITMC approach for UDA. As shown in Fig. 1, in each epoch, the UDA task is first performed by the teacher model. Subsequently, in the TKD module, we introduce a collaborative loss to help the teacher model $\theta_{T,e}$ distill the learned knowledge from both the source and target domains into the student model $\theta^0_{S,e}$. The student model then calculates the pruning rate and sorts the channels in the ACP module. The pruning is performed through multiple iterations until the target pruning rate of the current epoch is reached. After pruning in each iteration, TKD is used again to make the student model $\theta^1_{S,e}$ adapt to pruned structure and the student model in each iteration is updated as $\theta^2_{S,e}$. The above process is repeated multiple epochs until the student model is converged. In the following subsections, each module of the proposed training process will be introduced in detail.

3.2. Transfer Knowledge Distillation (TKD)

In this subsection, we propose a TKD approach to aid the distillation of knowledge from the teacher model into the student model by means of encompassing both the source and target domains. Additionally, a trade-off factor is introduced to balance the student model between fitting the source domain and adapting to the target domain.

3.2.1. Teacher model training of UDA

Prior to TKD, the UDA is performed for the teacher model. We follow the standard evaluation guidelines of UDA in [36], where all the training samples from the source domain are labeled and that from the target domain are unlabeled. The UDA loss of the teacher model consists of the MMD loss and cross-entropy loss [37]. Notably, MMD is commonly utilized in UDA to mitigate the distribution discrepancy between the source and target domains [28] and the loss function is defined as

$$L_{mmd} = \left\| \frac{1}{N_S}\sum_{i=1}^{N_S}\phi(x_i) - \frac{1}{N_T}\sum_{j=1}^{N_T}\phi(x_j) \right\|^2_{\mathcal{H}}, \tag{1}$$

where the feature mapping function $\phi(\cdot)$ maps the original data to a reproducing kernel hilbert space (RKHS); the RKHS is defined using the Gaussian kernel and denoted by $\mathcal{H}$; x_i and x_j represent the ith sample of the source domain and the jth sample of the target domain, respectively; N_S and N_T denote the numbers of samples in source domain and target domain, respectively.

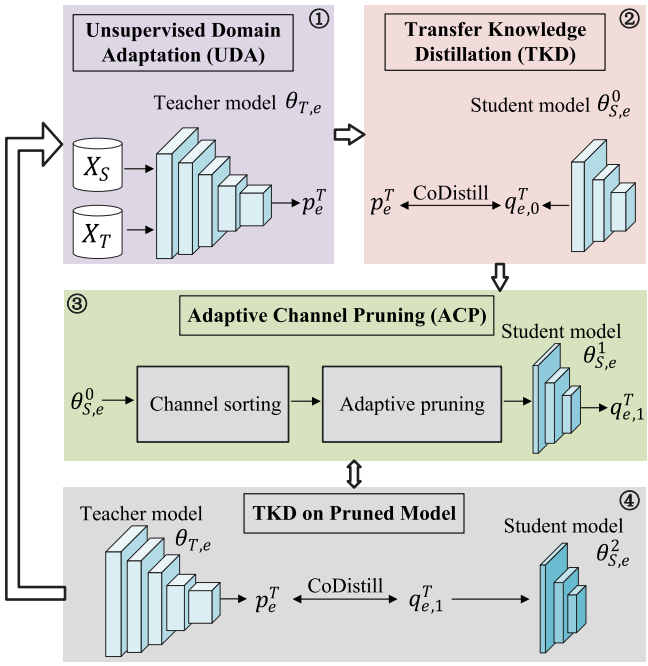


Fig. 1. The workflow of the proposed ITMC method.

Furthermore, the supervised cross-entropy loss in the source domain can be formulated as

$$L_{Tce}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = -\frac{1}{N_S}\sum_{i=1}^{N_S}\sum_{c=1}^{C} y_{ic}\log \hat{y}_{ic}, \tag{2}$$

where C is the number of classes; $\boldsymbol{y}$ and $\hat{\boldsymbol{y}}$ denote the labels and the predictions of training samples, respectively; y_{ic} and $\hat{y}_{ic}$ denote the true label and the prediction that the ith source domain sample belongs to class c, respectively. The cross-entropy loss function is employed to quantify the dissimilarity between these predictions and the ground truth labels. By minimizing this loss function, the model aims to align its predictions with the true labels.

Given (1) and (2), the total loss function for the teacher model is given by

$$L_T = L_{Tce}(\boldsymbol{y}, \hat{\boldsymbol{y}}) + \alpha L_{mmd}, \tag{3}$$

where α is a hyperparameter to balance the importance of the cross-entropy loss and the MMD loss [32].

3.2.2. Student model training of TKD

In spite of a more complex structure than the student model, the teacher model can effectively extract and represent information from both the source domain and the target domain. Hence, it is preferred to transfer the knowledge learned by the teacher model to the student model, rather than training the student model alone, especially when there is no labeled data available in the target domain.

By distilling the knowledge from the target domain, the student model gains a better understanding of the target domain data. Additionally, the knowledge distilling from the source domain ensures that the student model captures the shared features between the source and target domains. Hence, distilling the knowledge from both the source domain and target domain will enhance the student model's ability to adapt to varying domain.

Unlike the traditional KD method in [15], the primary objective of the proposed TKD is not to enhance the student model's performance on its original classification task. Instead, we focus on aligning the predictive distributions of the student model with those of the teacher model for both the source and target domains.

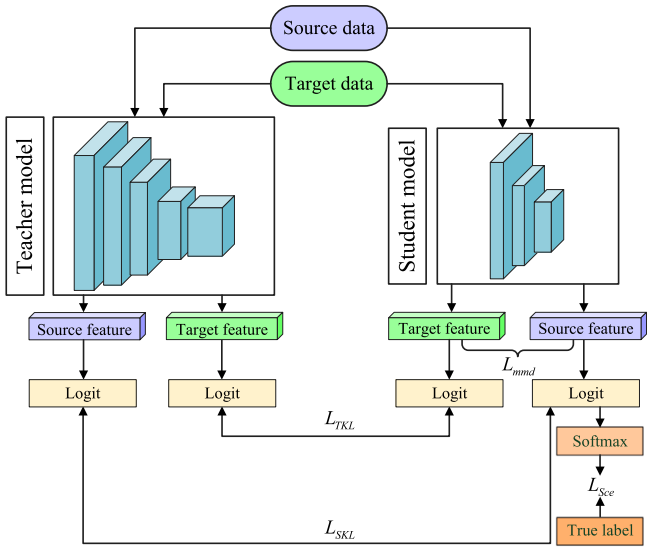


Fig. 2. The training architecture of TKD.

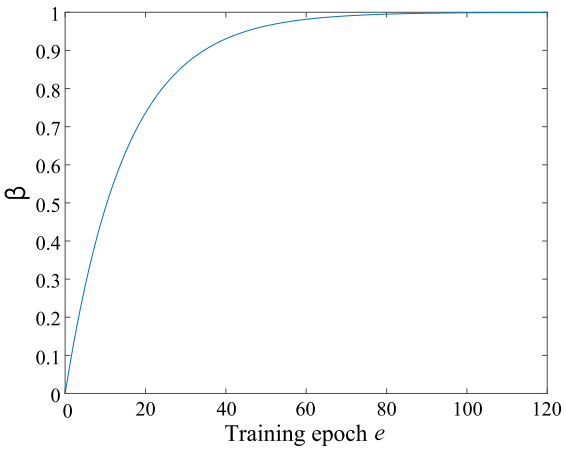


Fig. 3. Illustration of the trade-off factor β in (7).

As shown in Fig. 2, we devise a collaborative loss based on Kullback–Leibler (KL) divergence. This loss function quantifies the similarity between the outputs of the student and teacher models in both the source and target domains. By doing so, the student model is encouraged to maintain consistency across the two domains in order to facilitate the transfer of knowledge. The proposed collaborative loss function is given by

$$L_{KL}(\boldsymbol{p},\boldsymbol{q}) = L_{SKL} + L_{TKL} = -\sum_{i=1}^{N_S}\sum_{c=1}^{C} p_{ic}^{T}\log\frac{p_{ic}^{T}}{q_{ic}^{T}} - \sum_{j=1}^{N_T}\sum_{c=1}^{C} p_{jc}^{T}\log\frac{p_{jc}^{T}}{q_{jc}^{T}}, \quad (4)$$

where L_{SKL} and L_{TKL} denote the KL loss in the source domain and target domain, respectively; T is the temperature parameter used to soften the output of the model, which is generally greater than 1. Employing temperature scaling facilitates the transmission of more informative insights from the teacher model to the student model.

Note that $\boldsymbol{p}$ and $\boldsymbol{q}$ in (4) are the predicted probabilities of the teacher model and the student model, respectively, where p_{ic}^{T} (p_{jc}^{T}) and q_{ic}^{T} (q_{jc}^{T}) represent the probabilities that the teacher model and the student model predict that the source sample x_i (target sample x_j) belongs to class c at temperature T, respectively, i.e.,

$$p_{ic}^{T} = \frac{\exp(z_{ic}/T)}{\sum_{k=1}^{C}\exp(z_{ik}/T)},$$

$$p_{jc}^{T} = \frac{\exp(z_{jc}/T)}{\sum_{k=1}^{C}\exp(z_{jk}/T)},$$

$$q_{ic}^{T} = \frac{\exp(v_{ic}/T)}{\sum_{k=1}^{C}\exp(v_{ik}/T)},$$

$$q_{jc}^{T} = \frac{\exp(v_{jc}/T)}{\sum_{k=1}^{C}\exp(v_{jk}/T)}, \quad (5)$$

where z_{ic} (z_{jc}) and v_{ic} (v_{jc}) represent the output logits that the teacher model and the student model predict that source sample x_i (target sample x_j) belongs to class c, respectively.

In pursuit of performance enhancement, the student model is trained through the assimilation of knowledge from the teacher model and the utilization of labels available in the source domain. As a result, the overall loss function of the student model is given by

$$L_S = (1-\beta)L_{Sce} + \beta(L_{KL}(\boldsymbol{p},\boldsymbol{q}) + L_{mmd}), \quad (6)$$

where L_{Sce} is the supervised cross-entropy loss of the student model on the source domain. The KL losses L_{SKL} and L_{TKL} focus on matching the output distributions of the teacher model and the student model on both source domain and target domain. However, regrading cross-domain knowledge transfer tasks, discrepancies in data distributions between the source and target domains can adversely affect the model generalization. L_{mmd}, on the other hand, measures the distribution difference between the source and target domain features by calculating the distance between their means. Introducing L_{mmd} into the knowledge distillation can effectively align the feature distribution of the source domain with that of the target domain. Thus, the joint use of L_{KL} and L_{mmd} enables more comprehensive knowledge transfer from teacher model to student model. The KL loss ensures that the student model aligns with the teacher model in terms of category prediction, while the MMD loss ensures that the features learned by the student model are better adapted to the data distribution of the target domain.

Notably, the importance between the different losses of the student model in (6) is balanced by a trade-off factor β, given by

$$\beta = 1 - \exp(\frac{-8e}{N_e}), \quad (7)$$

where $e \in (0, N_e]$ and N_e is the total number of training epochs. Fig. 3 shows that β gradually increases from 0 to 1. Since UDA and TKD are performed alternately, the knowledge learned by the teacher model is limited in the early stage of training. In view of this, the student model places more importance on learning the source domain knowledge by itself at the beginning of training, and then gradually shifts the importance to the learning of the teacher model's knowledge. As the training continues, the student model can effectively reduce the domain drift, and its weight parameters gradually stabilize. In this context, Fig. 3 shows that β first sharply increases and then gradually stabilizes as the number of epochs goes large.

3.3. Adaptive Channel Pruning (ACP)

In order to further compress the student model, ACP is performed on the student model after TKD. To maintain the performance of the pruned model, we design an adaptive pruning formula that involves the number of training epochs. As shown in Fig. 4, during the ACP process, the pruning rate is calculated based on the adaptive pruning formula and the iteration counter p is initialize to 1. Then, the importance of the channels is ranked and the unimportant channels are iteratively pruned through N_p^e iterations until the desired pruning rate for this epoch is reached. Finally, the model after each pruning is finetuned by TKD to adapt the model weights to the pruned structures.

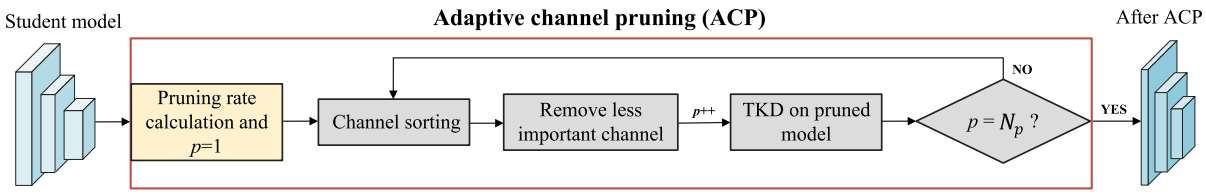


Fig. 4. The process of ACP. The redundant channels in the student model are iteratively pruned and finetuned by TKD.

3.3.1. Channel sorting criterion

The goal of channel sorting is to evaluate the importance of channels in each layer in order to prune insignificant channels. To assess the importance of each channel in the network, inspired by the work of [31], we utilize a channel importance assessment method based on the amount of loss change. Specifically, we measure the contribution of each channel to the model performance by comparing the amount of loss change of the model on the validation set before and after pruning. Intuitively, the larger amount of loss change leads to the greater the contribution of the pruned channel to the model and thereby the higher importance. We define the amount of loss change before and after pruning as

$$R(\boldsymbol{a}_i) = |L_S(\theta_S, \boldsymbol{a}_i) - L_S(\theta_S, \boldsymbol{a}_i = 0)|, \tag{8}$$

where $\boldsymbol{a}_i$ represents the activation values of the data, and θ_S represents the student model parameters.

However, it is computationally huge and impractical to find the optimal pruning scheme, i.e., to consider all combinations of channels simultaneously. Therefore, we adopt a greedy algorithm based on a first-order Taylor expansion to approximate the solution. Specifically, we consider the loss function of the model as a differentiable function with respect to the channel parameters. With the first-order Taylor expansion, we can approximate the change in the loss function after clipping a certain channel. The effectiveness of this approach has been verified in several studies. According to the Taylor expansion, the approximate expression of the function $f(x)$ at the point $x = b$ is

$$f(x) = \sum_{p=0}^{P} \frac{f^p(b)}{p!}(x-b)^p + R_p(x), \tag{9}$$

where $f^p(b)$ denotes the pth order derivative of the function $f(x)$ at the point $x = b$, and $R_p(x)$ denotes the Lagrange remainder term that represents the error that arises in the finite term expansion. Thus, the loss function $L_S(\theta_S, \boldsymbol{a}_i)$ in a particular case (i.e., $\boldsymbol{a}_i = 0$) is approximated as

$$L_S(\theta_S, \boldsymbol{a}_i = 0) = L_S(\theta_S, \boldsymbol{a}_i) - \frac{\partial L_S(\theta_S, \boldsymbol{a}_i)}{\partial \boldsymbol{a}_i} \cdot \boldsymbol{a}_i + R_1(\boldsymbol{a}_i = 0). \tag{10}$$

Due to the high cost of computing the Lagrange remainder term $R_1(\boldsymbol{a}_i = 0)$ exactly, we omit it to speed up the pruning process. Therefore, from (6) and (10), the channel evaluation criterion is given by

$$R(\boldsymbol{a}_i) = \left| (1-\beta)\frac{\partial L_{Sce}}{\partial \boldsymbol{a}_i^s}\boldsymbol{a}_i^s + \beta\left(\frac{\partial L_{KL}(\boldsymbol{p}, \boldsymbol{q})}{\partial \boldsymbol{a}_i^t}\boldsymbol{a}_i^t + \frac{\partial L_{mmd}}{\partial \boldsymbol{a}_i^t}\boldsymbol{a}_i^t\right)\right|. \tag{11}$$

where $i = 1, 2, \ldots, N_c^e$; N_c^e is the total number of channels of the student model in the eth epoch; $\boldsymbol{a}_i^s$ and $\boldsymbol{a}_i^t$ represent the activation values of the source domain and target domain data, respectively.

3.3.2. Adaptive channel pruning

According to the criterion in (11), we rank the importance of each channel in the student model. Based on this, we propose an adaptive pruning rule to progressively prune the least important channels of the model. The pruning rate of the eth epoch is given by

$$P(e) = \begin{cases} \frac{N_e^f - N_e^s}{60 N_e^2} e, & \text{if } N_e^s \le e < N_e^f \\ 0, & \text{otherwise,} \end{cases} \tag{12}$$

where $0 < N_e^s < N_e^f \le N_e$; N_e^s and N_e^f are the epochs that start and complete pruning, respectively.

The motivation of (12) is clarified as follows. First, in the early stage of training, pruning is not performed, since the student model needs to acquire sufficient knowledge and develop effective feature representations in the initial learning phase. During this period, the weight parameters of the student model are often unstable. Premature or excessive pruning could result in the loss of vital information, potentially degrading performance and hindering convergence. Then, as the number of training epochs increases and the student model stabilizes, a gradual increase of the pruning rate allows the model to become progressively simplified while retaining the performance. This dynamic adjustment prevents a sudden drop in the predictive performance of the student model.

Upon determining the pruning rate $P(e)$ in (12), the number of iterations for pruning in the eth epoch is given by

$$N_p^e = \frac{N_c^e \times P(e)}{n}, \tag{13}$$

where n is the number of channels to prune at each iteration, and there are total nN_p^e channels to be pruned in the eth epoch. Pruning is an irreversible operation, and once a channel is pruned, the lost connection cannot be recovered. Therefore, we adopt the strategy of pruning a small portion of channels at a time and repeat this process N_p^e iterations until the desired pruning rate $P(e)$ in (12) is reached in the current epoch.

After pruning n channels for each iteration, the model experiences a reduction in its expressive power. To tackle this issue, we further finetune the pruned student model again through TKD using (6). The main objective of TKD again is to adjust the weights and adapt them to the pruned structure.

3.4. Overview of the ITMC algorithm

The complete training process of the proposed ITMC approach is outlined in Algorithm 1. First, the teacher model performs UDA. Second, TKD with the loss L_S in (6) is employed to transfer the source and target domains knowledge from the teacher model to the student model. Third, the student model calculates the pruning rate in (12) and performs ACP. Then, the student model is finetuned by TKD again and updating the student model for the next iteration. The pruning process is repeated N_p^e times until the desired compression rate of the current epoch is met. Finally, the whole training process is repeated N_e epochs until the student model is converged.

Notably, conventional KD methods employ a well-trained teacher model while our proposed method optimizes the teacher model during training process. Nevertheless, our approach does not incur additional training overhead since the total number of training epochs for the pre-training based approach and our proposed approach is identical. Specifically, in pre-training based methods, the teacher model must complete training before it can serve as a mentor for the student model. In contrast, our method alternately trains the teacher model and the student model in each training epoch, and thereby the updates of the teacher model can be immediately fed back to the student model.

Algorithm 1: the ITMC Algorithm for UDA Model Compression

Input: A teacher model θ_T, a student model θ_S, source-labeled domain $X_S = \{(x_i, y_i)\}_{i=1}^{N_S}$, target-unlabeled domain $X_T = \{x_j\}_{j=1}^{N_T}$

Output: A pruned student model adapted for deep UDA

```
for e ← 1 to N_e do
    Optimize the teacher model θ_T by minimizing L_T in (3) and obtain the optimized model as θ_{T,e};
    Optimize the student model θ_S by minimizing L_S in (6) and obtain the optimized model as θ^0_{S,e};
    Calculate the pruning rate based P(e) in (12);
    Calculate the number of pruning iterations N^e_p in (13);
    for t ← 1 to N^e_p do
        if the target compression rate of the student model is achieved then
            break;
        else
            Sort the channels of the student model θ^0_{S,e} by the channel evaluation criterion in (11);
            Remove the n least important channels of θ^0_{S,e} and obtain a student model θ^1_{S,e};
            Finetune the pruned student model θ^1_{S,e} and update it as θ^2_{S,e} ;
        end
    end
end
```

4. Experiments

4.1. Datasets

4.1.1. Office-31

This dataset is commonly used for research on transfer learning. It consists of image data from three different domains: Amazon (A), Webcam (W), and DSLR (D). The Amazon domain contains product images from Amazon.com, while the Webcam and DSLR domains consist of images taken by webcams and high-quality images taken by digital SLR cameras respectively. Each domain contains 31 classes of target objects commonly found in office environments. However, the number of images varies across the domains. We employ six transfer scenarios to validate our ITMC method, i.e., A → W, A → D, W → D, W → A, D → W, D → A.

4.1.2. ImageCLEF-DA

This dataset serves as the benchmark dataset for ImageCLEF 2014 domain adaptation challenge, encompassing four distinct domains: Caltech-256 (C), Pascal VOC 2012 (P), ImageNet ILSVRC 2012 (I), and Bing (B). Each domain consists of 12 categories, with each category containing 50 images. Consistent with Office-31, six transfer tasks are simulated, i.e., C → P, C → I, P → C, P → I, I → C, I → P.

4.1.3. PACS

This is a domain-adaptive image dataset that contains 4 domains, Photo (P), Art painting (A), Cartoon (C) and Sketch (S). Each domain contains 7 categories inside it, and the 4 domains in this dataset have large domain differences between them. Similarly, we simulate six transfer learning scenarios, i.e., C → A, C → S, S → C, S → A, A → C, A → S.

4.2. Networks

4.2.1. VGG

The VGG [38] network is a classical convolutional neural network, whose main character is to use a small-sized convolutional kernel and small stride to build a deep network, which is able to extract local features and has a better generalization performance.

4.2.2. ResNet

The ResNet [39] network is a deep residual network, whose main feature is to use residual blocks to solve the gradient vanishing and exploding problems in the training of process, and is able to extract more complex features.

4.3. Baselines

We mainly compare the ITMC to the following baselines:

1. Applying channel pruning to the source domain task first, then fine-tune for the target domain task with the pruned model. (Direct pruning)
2. TCP-based UDA model compression [32].
3. TCP-based UDA model compression with CORAL.
4. PKD-based UDA model compression [34].
5. UNI-KD-based UDA model compression [35].
6. The UDA model is first pruned by TCP, followed by PKD.
7. The UDA model is first compressed by PKD, followed by TCP.

Note that the baselines maintain the same floating point operations (FLOPs) reduction rate as the proposed ITMC. For the KD module of all the methods, two sets of student-teacher model architectures, i.e., VGG16-VGG19 and ResNet50-ResNet101 are used, which can achieve 21% and 47% FLOPs reduction, respectively. For the pruning module, pruning two student models, i.e., VGG16 and ResNet50, achieves 26% and 12% decrease in FLOPs, respectively. Specifically, for the VGG16 network, we prune the channels of the convolutional layer and the first fully connected layer associated with the convolutional layer. For the ResNet50 network, considering its complex structure, we prune the convolutional layers without dimension dependency.

4.4. Implementation details

All methods are implemented based on PyTorch framework with an NVIDIA GeForce RTX 4090 GPU. The training starts with pre-trained models on ImageNet dataset. During the training, all the images are cropped to a fixed size of 224 × 224 and the training batch size is 32. For TKD, we set the distillation temperature $T = 20$. In the ACP process, n is set to 16, which means that 16 channels are pruned at each iteration. All the networks are trained using stochastic gradient descent (SGD) method with a momentum of 0.9. The learning rate starts from 0.001, and gradually reduces to ensure a stable training. Finally, we use the classification accuracy and the FLOPs of the model as evaluation metrics.

4.5. Visualization analysis

To visually compare the performance of different methods in the target domain, the extracted features of task A→W from different methods (i.e., TCP, PKD, UNI-KD, PKD→TCP, TCP→PKD and ITMC) are illustrated in Fig. 5 using t-distributed stochastic neighbor embedding (t-SNE) technique, where different colors represent extracted features of different classes. Here we randomly pick 10 classes in domain W of Office-31 dataset to visualize, and the number in each cluster indicates the index of the class corresponding to these features.

It can be seen that the proposed ITMC method clusters the features well under the same task with clear boundaries between target categories. In contrast, the features of other methods are more scattered.

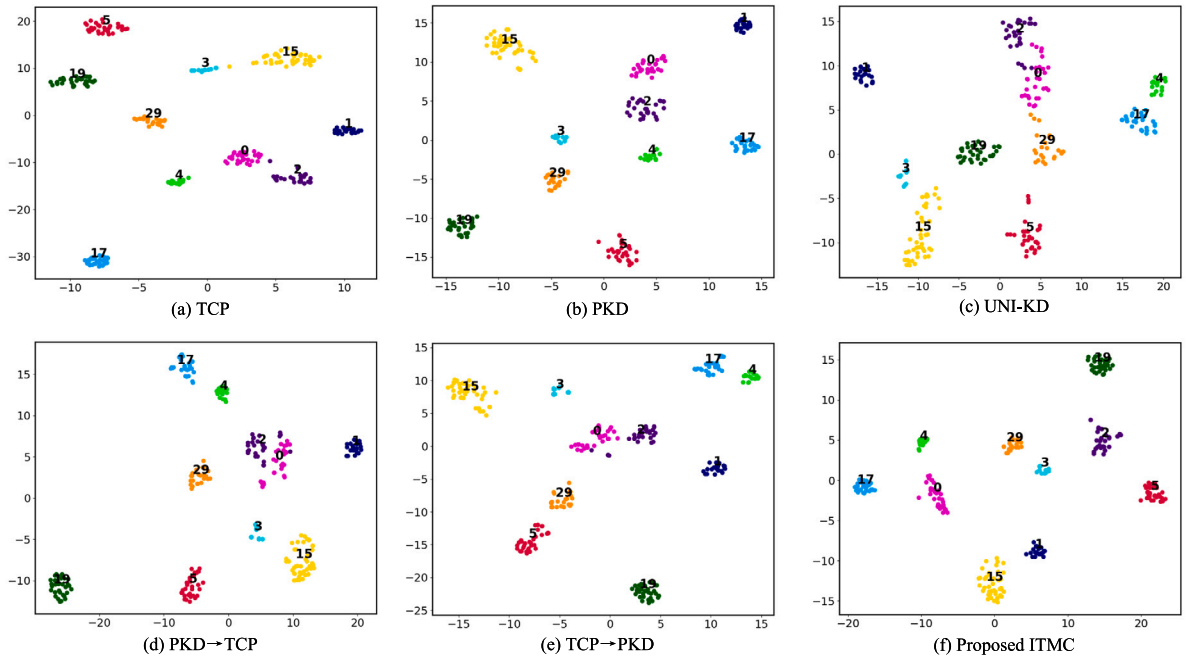


Fig. 5. The t-SNE visualization of features extracted by ResNet50 in task A→W on Office-31 dataset.

Table 1
The accuracy (%) with VGG16 as the student model on Office-31 dataset.

Methods	D→W	W→D	W→A	A→W	A→D	D→A	Average
VGG16-base	94.0	97.5	55.2	74.0	72.3	54.1	74.5
Direct pruning	94.5	99.0	47.2	69.4	69.8	42.7	70.4
TCP [32]	96.1	99.8	51.2	76.1	76.2	47.9	74.6
TCP_CORAL	95.2	99.1	50.0	71.3	70.7	50.5	72.8
PKD [34]	96.9	**100.0**	53.3	66.5	70.5	54.2	73.6
UNI-KD [35]	95.6	**100.0**	53.1	68.5	69.9	55.2	73.7
PKD→TCP	94.0	98.2	45.3	64.2	66.3	47.3	69.2
TCP→PKD	95.6	99.6	46.8	69.2	73.1	46.9	71.9
ITMC	**97.8**	**100.0**	**55.4**	74.0	74.1	**56.3**	**76.3**

Table 2
The accuracy (%) with ResNet50 as the student model on Office-31 dataset.

Methods	D→W	W→D	W→A	A→W	A→D	D→A	Average
ResNet50-base	97.1	99.2	62.3	80.3	78.9	64.3	80.3
Direct pruning	96.7	99.5	50.1	75.8	76.0	48.0	74.4
TCP [32]	98.2	99.8	55.5	81.8	77.9	50.0	77.2
TCP_CORAL	91.6	99.3	59.2	79.4	78.1	50.6	76.4
PKD [34]	96.2	99.4	58.5	78.2	79.5	50.8	77.1
UNI-KD [35]	94.1	**100.0**	60.2	75.3	76.3	56.1	77.0
PKD→TCP	96.7	99.2	53.8	79.1	77.7	52.8	76.6
TCP→PKD	93.6	99.6	53.7	79.6	76.5	49.7	75.5
ITMC	**98.5**	**100.0**	**62.6**	**84.6**	**82.7**	**60.9**	**81.6**

For example, Fig. 5(c), (d) and (e) show that the feature points of class "0" and class "2" are dispersed and some of them overlap which are hard to be classified. This suggests that ITMC can effectively extract features and thus improve the classification accuracy.

4.6. Results and analysis

4.6.1. Office-31 dataset

The accuracy comparisons between different methods on Office-31 dataset are shown in Tables 1 and 2. First, it is observed that the accuracy can be improved by up to 11.2% compared with all the baselines with ResNet50 model in the D→A task.

Second, it is observed that the proposed method outperforms TCP, PKD and UNI-KD for most transfer learning tasks. This is due to the fact

Table 3
The accuracy (%) with VGG16 as the student model on ImageCLEF-DA dataset.

Methods	C→I	C→P	P→C	P→I	I→P	I→C	Average
VGG16-base	77.0	61.1	87.2	80.0	71.3	88.5	77.5
Direct pruning	64.5	56.0	83.3	77.7	68.0	88.5	73.0
TCP [32]	77.8	64.8	87.5	80.5	72.0	90.5	78.9
TCP_CORAL	73.2	61.2	82.4	79.1	72.6	89.0	76.3
PKD [34]	75.0	63.8	86.8	82.2	**73.3**	91.3	78.7
UNI-KD [35]	78.6	65.3	87.6	83.4	72.8	91.0	79.8
PKD→TCP	66.2	55.5	81.8	75.3	67.2	84.7	71.8
TCP→PKD	66.7	56.7	80.0	80.7	67.8	87.8	73.3
ITMC	**81.0**	**68.0**	**89.7**	**85.3**	73.0	**92.7**	**81.6**

Table 4
The accuracy (%) with ResNet50 as the student model on ImageCLEF-DA dataset.

Methods	C→I	C→P	P→C	P→I	I→P	I→C	Average
ResNet50-base	83.3	70.0	89.8	82.2	74.8	92.3	82.1
Direct pruning	76.5	64.0	84.0	81.3	71.8	92.1	78.2
TCP [32]	80.8	66.2	86.5	82.6	75.0	92.5	80.6
TCP_CORAL	77.4	65.1	80.9	82.2	73.6	90.4	78.3
PKD [34]	76.5	67.2	85.8	83.3	72.7	92.8	79.7
UNI-KD [35]	84.1	68.5	88.6	85.8	74.1	92.0	82.2
PKD→TCP	81.8	67.2	87.5	82.7	71.2	91.2	80.3
TCP→PKD	76.2	63.3	87.0	83.0	70.2	93.1	78.8
ITMC	**85.8**	**70.2**	**92.3**	**90.7**	**78.2**	**94.7**	**85.3**

Table 5
The accuracy (%) with VGG16 as the student model on PACS dataset.

Methods	C→A	C→S	S→C	S→A	A→C	A→S	Average
VGG16-base	60.9	65.2	63.3	57.4	67.7	63.1	62.9
Direct pruning	59.0	60.1	54.2	53.3	60.6	58.3	57.6
TCP [32]	60.8	62.7	62.1	58.4	63.5	62.0	61.6
TCP_CORAL	61.0	61.8	65.5	56.2	65.2	61.6	61.9
PKD [34]	61.3	63.8	64.0	56.2	64.8	63.2	62.2
UNI-KD [35]	60.4	64.4	64.3	60.7	68.0	63.9	63.6
PKD→TCP	58.2	63.5	56.9	60.1	66.9	60.1	61.0
TCP→PKD	57.7	65.0	58.7	58.5	67.3	61.3	61.4
ITMC	**62.5**	**68.1**	**66.4**	**61.7**	**70.2**	**65.9**	**65.8**

Table 6
The accuracy (%) with ResNet50 as the student model on PACS dataset.

Methods	C→A	C→S	S→C	S→A	A→C	A→S	Average
ResNet50-base	72.2	78.6	72.3	66.2	79.1	66.5	72.5
Direct pruning	66.4	76.7	68.3	65.1	67.6	61.1	67.5
TCP [32]	70.8	80.5	70.5	66.3	77.7	62.8	71.4
TCP_CORAL	69.5	79.2	70.6	68.4	72.3	63.5	70.6
PKD [34]	69.9	81.3	71.6	68.7	69.4	65.6	71.1
UNI-KD [35]	75.8	79.9	69.2	70.0	78.9	69.9	74.0
PKD→TCP	71.1	77.1	68.4	65.9	73.6	68.2	70.7
TCP→PKD	74.2	76.6	70.3	66.4	71.8	67.8	71.2
ITMC	**79.3**	**83.5**	**75.0**	**70.8**	**81.4**	**79.7**	**78.3**

that we jointly optimize the two compression methods, wherein TKD guides the student model to improve the accuracy, while ACP improves the generalization ability of the model by gradually pruning the channels. Compared with TCP, the results demonstrate that the proposed adaptive pruning strategy and TKD can improve the performance of the student model in the target domain. Compared with PKD and UNI-KD, the proposed ITMC method further reduces the redundancy of the student model to help create a more compact student model.

Third, we observe that the proposed method is also superior to the straightforward combinations of PKD and TCP, including PKD→TCP and TCP→PKD. This is attributed to the fact that, in the two-stage baselines, each stage utilizes its own loss function which emphasizes distinct aspects of the task [40]. In TCP→PKD, the pruned student model is not sufficient to learn the teacher model knowledge after TCP. In PKD→TCP, the student model performs TCP without the guidance of the teacher's model, which makes it difficult to capture the complex features in the data and ultimately leads to the poor accuracy.

4.6.2. ImageCLEF-DA dataset

The accuracy results of different methods on ImageCLEF-DA dataset are shown in Tables 3 and 4. Similar to Office-31 dataset, we observe that the proposed method achieves higher accuracy than the baselines. For example, in the C→I transfer task with VGG16 as the student model, the proposed method improves the accuracy by about 5% compared with the TCP and PKD, and about 14% compared with the straightforward combinations of PKD and TCP. Additionally, in the P→I transfer task with ResNet50 as the student model, the proposed method improves accuracy by up to 8.1% compared with all the baselines.

Therefore, KD and pruning should be seamlessly orchestrated during the training instead of treating them as two independent phases. Using the proposed ITMC, the complementary strengths of the two techniques are maximized to balance the student model size and the performance in the target domain.

4.6.3. PACS dataset

As shown in Tables 5 and 6, we conduct comparative experiments on the PACS dataset. The experimental results show that the ITMC method achieves higher accuracy in most transfer tasks compared with TCP, PKD, and a baseline method that sequentially combines these two methods. For example, in the A→C task with VGG16 as the student model, the ITMC method outperforms TCP and PKD methods by 6.7% improvement, PKD→TCP and TCP→PKD by 9.5% improvement, and direct pruning by 12.2% improvement. Similarly, in the A→S task with ResNet50 as the student model, the accuracy of the proposed method is improved by up to 18.6% compared with all the baselines. These experimental results fully demonstrate that the ITMC method significantly improves model performance on multiple datasets and transfer tasks, which corroborate its effectiveness and robustness in dealing with cross-domain transfer learning problems.

4.6.4. Convergence analysis of alternate execution of TKD and ACP

To visualize the convergence of the model training process, we present the accuracy and loss of ResNet50 in each training epoch for I→P task in ImageCLEF-DA and W→D task of Office-31. As shown in Fig. 6, the continuous improvement of the accuracy and the steady decrease of the loss indicate that the model is continuously optimized and adjusted during the training process. TKD enhances the generalization ability of the student model through knowledge distillation, while ACP reduces the model complexity through progressive pruning. The iterative operation of TKD and ACP can enhance the convergence efficiency of the model. The smoothness of the training process indicates that this alternating strategy does not introduce any instability, and further corroborates its effectiveness and reliability in model optimization.

4.7. Effect of FLOPs reduction in ITMC

Fig. 7 compares the Top-1 accuracy between our ITMC method and the baseline methods in the D→A transfer task under different FLOPs reduction rates by adjusting N_e^f, i.e., changing the epoch that complete pruning. Compared with the baselines, Fig. 7(a) and Fig. 7(b) show that the proposed ITMC achieves the higher accuracy under different FLOPs reduction rates based on both VGG16 and ResNet50 networks. In addition, the accuracy of the proposed ITMC changes more smoothly as the FLOPs reduction rate goes up in the two networks. These observations corroborate the effectiveness of the ITMC method.

4.8. Ablation experiments

To corroborate the effectiveness of the TKD and ACP modules of the proposed method, we conduct the following ablation experiments.

4.8.1. Effect of trade-off factor in TKD

The trade-off factor in (7) is used to balance between fitting labeled data in the source domain and adapting unlabeled data in the target domain. As shown in Fig. 8, the importance of β is evaluated in 6 transfer tasks of Office-31 dataset using two networks, ResNet50 and VGG16. It is observed that dynamic adjustment of the trade-off factor β can achieve the higher accuracy than the fixed β for all the transfer tasks.

In the training of the teacher model, the setting of β directly affects the learning effect of the student model. At the initial stage, the knowledge transferred from the teacher model has not yet achieved a high level of reliability, and a higher value of β may cause the student model to converge to a suboptimal solution. Therefore, a gradual increase of β facilitates a progressive alignment of the student model with the knowledge of the teacher model throughout the learning process, thereby enhancing the generalization capability of the student model.

4.8.2. Effect of KL loss (L_{TKL}) on TKD in the target domain

To deeply explore the impact of L_{TKL} in (6) on model performance, Table 7 presents the results of ablation experiments on the Office-31 dataset with two classical network architectures, i.e., VGG16 and ResNet50. The experimental results show that the model performance decreases significantly without the aid of knowledge distillation in the target domain. The reason is that if we only distill the source domain knowledge, the student model is difficult to learn the categorical information that is shared between the source and target domains. The introduction of L_{TKL} contributes to a significant improvement of the model performance in the target domain. This suggests that the teacher model can guide the student model to learn a data distribution that is more suitable for the target domain with L_{TKL}.

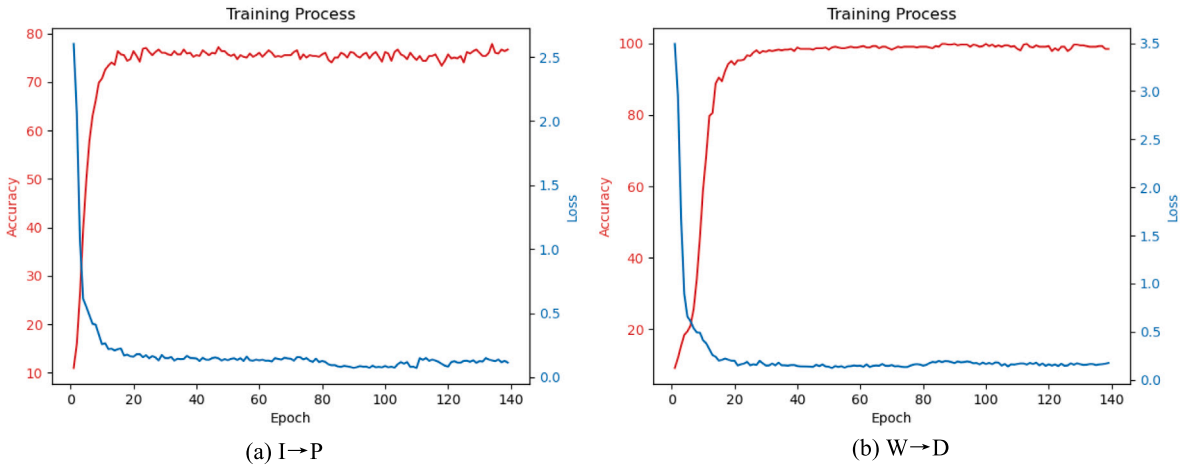


Fig. 6. Training process in different transfer tasks based on ResNet50.

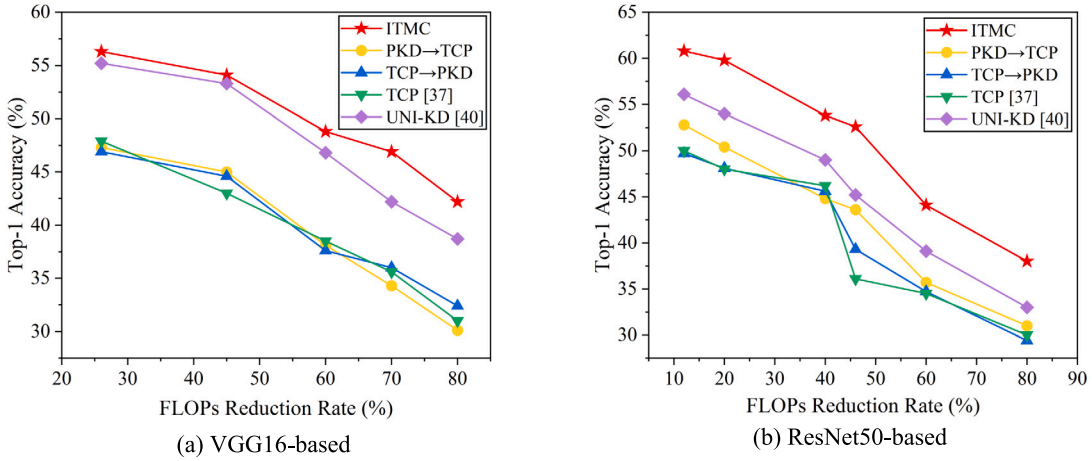


Fig. 7. Top-1 accuracy comparison between the proposed ITMC and the baselines under different reduction of FLOPs in the transfer scenarios D→A on Office-31 Dataset.

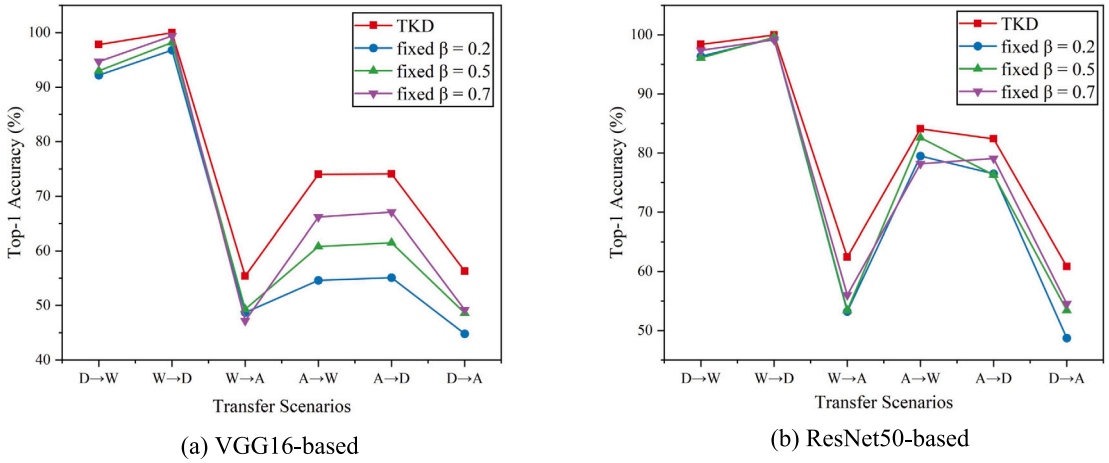


Fig. 8. Top-1 accuracy comparison between the proposed TKD and fixed β under $\beta = 0.2, 0.5, 0.7$ on Office-31 dataset.

4.8.3. *Effect of MMD loss (L_{mmd}) on TKD*

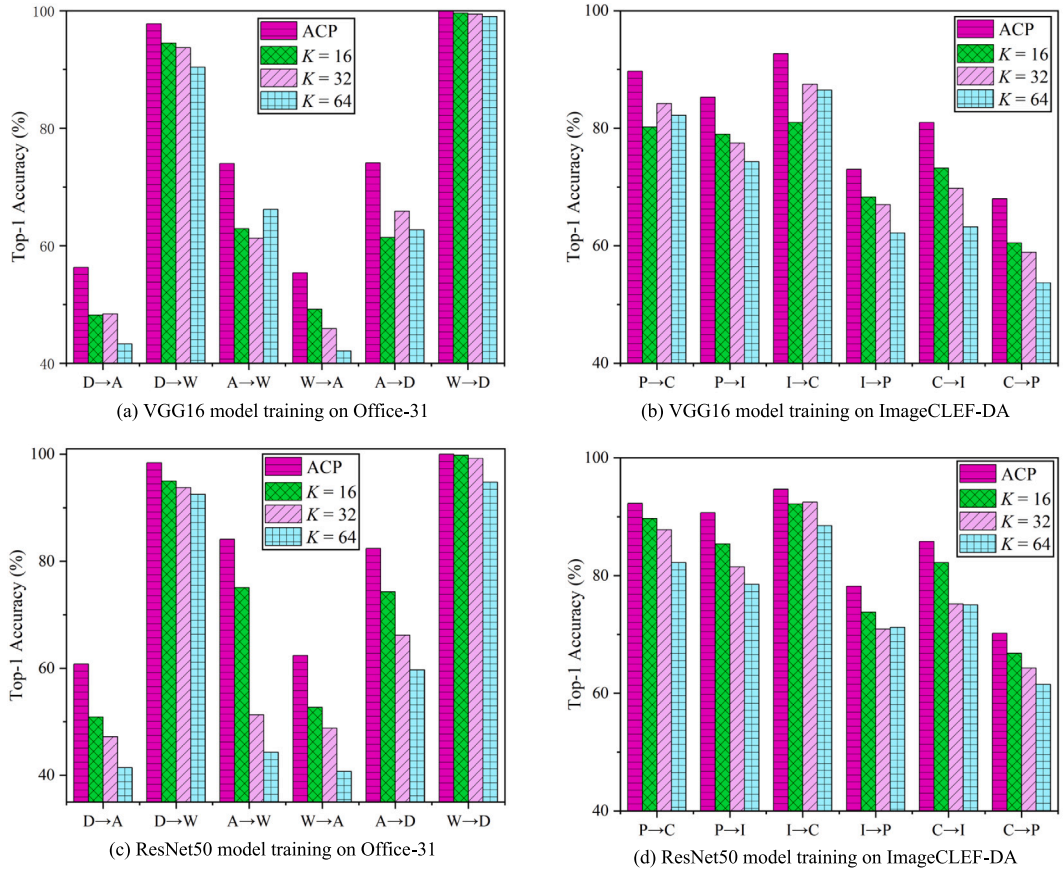
To investigate the effect of L_{mmd} in (6) on the model performance, we conduct an ablation experiment based on VGG16 and ResNet50 on the Office-31 dataset. The experimental results in Table 8 show that the model performance on the target domain is degraded without the aid of L_{mmd}. This is mainly due to the fact that, although the knowledge distillation method can effectively transfer the categorical knowledge from the teacher model to the student model, it ignores the distributional differences between the source and target domain data at the feature level. Since L_{mmd} can effectively measure the difference between two distributions, minimizing the MMD distance between features in the source and target domains helps align their distributions.

Table 7
Results of removing L_{TKL} on the Office-31 dataset (%).

Network	Methods	D→W	W→D	W→A	A→W	A→D	D→A	Average
VGG16	ITMC w/o L_{TKL}	95.1	97.6	51.9	73.2	72.8	53.0	73.9
	ITMC	**97.8**	**100.0**	**55.4**	**74.0**	**74.1**	**56.3**	**76.3**
ResNet50	ITMC w/o L_{TKL}	95.9	98.1	58.9	80.4	80.8	58.2	78.7
	ITMC	**98.5**	**100.0**	**62.6**	**84.6**	**82.7**	**60.9**	**81.6**

Table 8
Results of removing L_{mmd} on the Office-31 dataset (%).

Network	Methods	D→W	W→D	W→A	A→W	A→D	D→A	Average
VGG16	ITMC w/o L_{mmd}	96.0	98.1	52.8	71.7	72.0	54.3	74.2
	ITMC	**97.8**	**100.0**	**55.4**	**74.0**	**74.1**	**56.3**	**76.3**
ResNet50	ITMC w/o L_{mmd}	94.3	97.4	55.9	81.7	79.6	57.0	77.7
	ITMC	**98.5**	**100.0**	**62.6**	**84.6**	**82.7**	**60.9**	**81.6**



(a) VGG16 model training on Office-31
(b) VGG16 model training on ImageCLEF-DA
(c) ResNet50 model training on Office-31
(d) ResNet50 model training on ImageCLEF-DA

Fig. 9. Top-1 accuracy comparison between the proposed ACP and fixed channel pruning under $K = 16, 32, 64$.

This alignment enables the student model to learn more robust feature representations, thereby enhancing its generalization ability in the target domain.

4.8.4. Effect of pruning strategy in ACP

The pruning rate of (12) plays a crucial role in reducing the model size. We further study the impact of the pruning rate $P(e)$ on the performance by comparing the proposed $P(e)$ with a fixed pruning of K channels in each epoch. As shown in Fig. 9, the performance of the proposed ACP strategy and fixed channel pruning are evaluated on different datasets using different networks. It is observed that in contrast to pruning a fixed number of channels per epoch, the student model achieves the higher accuracy under ACP. In addition, the accuracy drops sharply if the fixed number of pruned channels is large, e.g., $K = 64$. This is due to the fact that the weight parameters of the student model are unstable at the beginning of training, and pruning excessive channels may weaken the model's ability to represent critical task-specific features, ultimately leading to performance degradation.

5. Conclusion

In this paper, we have proposed ITMC to effectively compress the model for UDA. Specifically, a collaborative loss function has been developed in the TKD module, which enables the student model to learn feature representations from both the source and target domains. Moreover, a novel ACP module has been designed to iteratively reduce the redundancy of the student model and thereby prevent sudden drop in accuracy during training. The alternation of TKD and ACP ensures effective knowledge transfer, balancing the model size and its performance in the target domain. Extensive experiments have been

conducted on various network architectures and datasets to verify the effectiveness of the proposed ITMC. Compared with the state-of-the-art methods, the proposed ITMC has achieved the higher classification accuracy under the same compression ratio.

The proposed ITMC method demonstrates potential in the field of UDA; however, its applicability is predominantly confined to CNN, as the channel-based pruning technique is exclusively designed for convolutional layers. In future research, we plan to generalize the model compression technique to network architectures based on self-attention mechanisms, such as Transformer. In addition, we will explore the integration of self-supervised learning with model compression to further augment the generalization ability of UDA models.

CRediT authorship contribution statement

Zhiyuan Wang: Writing – original draft, Validation. **Long Shi:** Writing – review & editing. **Zhen Mei:** Writing – review & editing, Validation. **Xiang Zhao:** Writing – review & editing. **Zhe Wang:** Writing – review & editing. **Jun Li:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

The authors are Editorial Board Members for [IEEE Transactions on Wireless Communication, IEEE Transactions on Cognitive Communications and Networking] and were not involved in the editorial review or the decision to publish this article.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (No. 62371239, No. 62201258, No. 62272469, No. 62202232), in part by the Natural Science Foundation of Jiangsu Province under Grant BK20210331, in part by the Jiangsu Specially-Appointed Professor Program 2021, in part by “the Fundamental Research Funds for the Central Universities”, No.30923011035, in part by the Science and Technology Innovation Program of Hunan Province (No. 2023RC1007).

Data availability

The authors do not have permission to share data.

References

[1] Z. Zou, K. Chen, Z. Shi, Y. Guo, J. Ye, Object detection in 20 years: A survey, Proc. IEEE 111 (3) (2023) 257–276.
[2] K. Chowdhary, K. Chowdhary, Natural language processing, Fundam. Artif. Intell. (2020) 603–649.
[3] Y. Zhang, W. Chan, N. Jaitly, Very deep convolutional networks for end-to-end speech recognition, in: 2017 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, 2017, pp. 4845–4849, http://dx.doi.org/10.1109/ICASSP.2017.7953077.
[4] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, Q. He, A comprehensive survey on transfer learning, Proc. IEEE 109 (1) (2020) 43–76.
[5] H. Tang, K. Jia, Vicinal and categorical domain adaptation, Pattern Recognit. 115 (2021) 107907.
[6] X. Liu, C. Yoo, F. Xing, H. Oh, G. El Fakhri, J.-W. Kang, J. Woo, et al., Deep unsupervised domain adaptation: A review of recent advances and perspectives, APSIPA Trans. Signal Inf. Process. 11 (1) (2022).
[7] Y. Cheng, D. Wang, P. Zhou, T. Zhang, A survey of model compression and acceleration for deep neural networks, 2017, arXiv preprint arXiv:1710.09282.
[8] Y. Xu, Y. Wang, A. Zhou, W. Lin, H. Xiong, Deep neural network compression with single and multiple level quantization, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, 2018, 1.
[9] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, X.-s. Hua, Quantization networks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 7308–7316.
[10] S. Vadera, S. Ameen, Methods for pruning deep neural networks, IEEE Access 10 (2022) 63280–63300.
[11] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, J. Kautz, Importance estimation for neural network pruning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 11264–11272.
[12] Y. He, X. Zhang, J. Sun, Channel pruning for accelerating very deep neural networks, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 1389–1397.
[13] Y. Idelbayev, M.A. Carreira-Perpinán, Low-rank compression of neural nets: Learning the rank of each layer, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 8049–8059.
[14] M. Jaderberg, A. Vedaldi, A. Zisserman, Speeding up convolutional neural networks with low rank expansions, 2014, arXiv preprint arXiv:1405.3866.
[15] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, 2015, arXiv preprint arXiv:1503.02531.
[16] A. Romero, N. Ballas, S.E. Kahou, A. Chassang, Y. Bengio, FitNets: Hints for thin deep nets, in: ICLR, 2015.
[17] G. Xu, Z. Liu, C.C. Loy, Computation-efficient knowledge distillation via uncertainty-aware mixup, Pattern Recognit. 138 (2023) 109338.
[18] S.W. Prakosa, J.-S. Leu, Z.-H. Chen, Improving the accuracy of pruned network using knowledge distillation, Pattern Anal. Appl. 24 (2021) 819–830.
[19] J. Park, A. No, Prune your model before distill it, in: European Conference on Computer Vision, Springer, 2022, pp. 120–136.
[20] N. Aghli, E. Ribeiro, Combining weight pruning and knowledge distillation for cnn compression, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 3191–3198.
[21] S. Han, H. Mao, W.J. Dally, Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding, in: 4th International Conference on Learning Representations, 2016.
[22] A. Polino, R. Pascanu, D. Alistarh, Model compression via distillation and quantization, 2018, arXiv preprint arXiv:1802.05668.
[23] Z. Chen, Z. Chen, J. Lin, S. Liu, W. Li, Deep neural network acceleration based on low-rank approximated channel pruning, IEEE Trans. Circuits Syst. I. Regul. Pap. 67 (4) (2020) 1232–1244.
[24] Z. Wang, L. Shi, Z. Mei, X. Zhao, Z. Wang, J. Li, Iterative transfer knowledge distillation and channel pruning for unsupervised cross-domain compression, in: International Conference on Web Information Systems and Applications, Springer, 2024, pp. 3–15.
[25] Z. Zhao, Q. Zhang, X. Yu, C. Sun, S. Wang, R. Yan, X. Chen, Applications of unsupervised deep transfer learning to intelligent fault diagnosis: A survey and comparative study, IEEE Trans. Instrum. Meas. 70 (2021) 1–28.
[26] W. Dai, Q. Yang, G.-R. Xue, Y. Yu, Boosting for transfer learning, in: Proceedings of the 24th International Conference on Machine Learning, 2007, pp. 193–200.
[27] Z. Mei, K. Cai, L. Shi, J. Li, L. Chen, K.A.S. Immink, Deep transfer learning-based detection for flash memory channels, IEEE Trans. Commun. (2024).
[28] K.M. Borgwardt, A. Gretton, M.J. Rasch, H.-P. Kriegel, B. Schölkopf, A.J. Smola, Integrating structured biological data by kernel maximum mean discrepancy, Bioinformatics 22 (14) (2006) 49–57.
[29] B. Sun, K. Saenko, Deep coral: Correlation alignment for deep domain adaptation, in: ECCV, Springer, 2016, pp. 443–450.
[30] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, V. Lempitsky, Domain-adversarial training of neural networks, J. Mach. Learn. Res. 17 (59) (2016) 1–35.
[31] Y. Ma, X. Yao, R. Chen, R. Li, X. Shen, B. Yu, Small is beautiful: Compressing deep neural networks for partial domain adaptation, IEEE Trans. Neural Netw. Learn. Syst. (2022).
[32] C. Yu, J. Wang, Y. Chen, Z. Wu, Accelerating deep unsupervised domain adaptation with transfer channel pruning, in: 2019 International Joint Conference on Neural Networks, IJCNN, IEEE, 2019, pp. 1–8.
[33] S. Zhang, L. Jiang, J. Tan, Cross-domain knowledge distillation for text classification, Neurocomputing 509 (2022) 11–20.
[34] E. Granger, M. Kiran, J. Dolz, L.-A. Blais-Morin, et al., Joint progressive knowledge distillation and unsupervised domain adaptation, in: International Joint Conference on Neural Networks, IJCNN, IEEE, 2020, pp. 1–8.
[35] Q. Xu, M. Wu, X. Li, K. Mao, Z. Chen, Distilling universal and joint knowledge for cross-domain model compression on time series data, in: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence Organization, 2023, pp. 4460–4468.
[36] B. Gong, K. Grauman, F. Sha, Connecting the dots with landmarks: Discriminatively learning domain-invariant features for unsupervised domain adaptation, in: International Conference on Machine Learning, PMLR, 2013, pp. 222–230.
[37] Y. Ho, S. Wookey, The real-world-weight cross-entropy loss function: Modeling the costs of mislabeling, IEEE Access 8 (2019) 4806–4813.
[38] S. Tammina, Transfer learning using vgg-16 with deep convolutional neural network for classifying images, Int. J. Sci. Res. Publ. (IJSRP) 9 (10) (2019) 143–150.
[39] S. Mascarenhas, M. Agarwal, A comparison between VGG16, VGG19 and ResNet50 architecture frameworks for image classification, in: 2021 International Conference on Disruptive Technologies for Multi-Disciplinary Research and Applications, CENTCON, vol. 1, IEEE, 2021, pp. 96–99.

[40] Q. Wang, Y. Ma, K. Zhao, Y. Tian, A comprehensive survey of loss functions in machine learning, Ann. Data Sci. (2020) 1–26.

Zhiyuan Wang received the B.S. degree in communication engineering from the Zhejiang Gongshang University, Hangzhou, China, in 2022, where she is currently pursuing the M.S. degree with the School of Electronic and Optical Engineering. Her current research interests include machine learning and model compression.

Long Shi received the Ph.D. degree in Electrical Engineering from the University of New South Wales, Sydney, Australia, in 2012. Now he is a Professor at the School of Electronic and Optical Engineering, Nanjing University of Science and Technology, Nanjing, China. His research interests include wireless communications, blockchain networks, and federated learning.

Zhen Mei received the Ph.D. degree from Newcastle University, U.K., in 2017. He is currently an Associate Professor at Nanjing University of Science and Technology, China. His research interests include machine learning, information and coding theory for data storage and communications systems, and industrial AI. He was honored as the Wiley Outstanding Open Science Author of the Year 2022.

Xiang Zhao received the Ph.D. degree from The University of New South Wales, Australia, in 2013. He is currently a professor at National University of Defense Technology, China. His research interests include advanced data analytics, with a special focus on graph data management and mining.

Zhe Wang received the Ph.D. degree in electrical engineering from The University of New South Wales, Sydney, Australia, in 2014. She is currently a professor with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China. Her research interests include applications of optimization, game theory, and machine learning to resource allocation in communications and networking.

Jun Li received Ph.D. degree in Electronic Engineering from Shanghai Jiao Tong University, Shanghai, P.R. China in 2009. He is currently a Professor at the School of Electronic and Optical Engineering, Nanjing University of Science and Technology, Nanjing, China. His research interests include network information theory, game theory, distributed intelligence, multiple agent reinforcement learning, mobile edge computing, network privacy and security, and industrial Internet of things.